

Smart Budget E-Commerce System

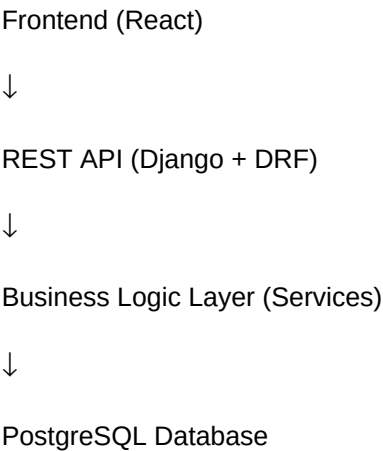
A full-stack Smart Budget Monitoring System designed for e-commerce platforms (like Amazon-style systems) that helps users control their monthly spending with real-time analytics and budget enforcement.

■ Project Overview

Online shoppers often overspend during sales and promotional events. This project introduces **Smart Budget Mode**, allowing users to:

- Set monthly spending limits
- Track real-time expenses
- Receive warnings at 80% usage
- Block checkout when budget is exceeded
- View spending analytics via dashboard

■■ System Architecture



■■ Tech Stack

■ *Backend*

- Django
- Django REST Framework
- JWT Authentication (SimpleJWT)
- PostgreSQL

■ *Frontend*

- React.js
- Axios (API calls)
- Basic UI Components

■ *DevOps*

- Docker
- docker-compose
- Environment Variables (.env)

■■ Features

- User Registration & Login (JWT Based)
- Set Monthly Budget
- Create Orders
- Real-time Budget Calculation
- Warning at 80% Limit

- Budget Exceeded Restriction
- RESTful API Architecture
- Modular Django App Structure
- Dockerized Setup

■ Project Structure

```
smart-budget-ecommerce/
├──
│   ├── backend/
│   │   ├── config/
│   │   ├── apps/
│   │   │   ├── users/
│   │   │   ├── budget/
│   │   │   └── orders/
│   │   ├── manage.py
│   │   └── requirements.txt
│   ├── frontend/
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── pages/
│   │   │   └── services/
│   │   └── package.json
│   ├──
│   ├── docker-compose.yml
│   └── README.md
```

■ Authentication Flow

1. User registers
2. User logs in
3. JWT access & refresh tokens generated
4. All protected APIs require Bearer Token

Example Header:

Authorization: Bearer <your_token>

■ API Endpoints

■ *Authentication*

Method	Endpoint	Description
--------	----------	-------------

-----	-----	-----
-------	-------	-------

POST	/api/register/	Register new user
------	----------------	-------------------

POST	/api/login/	Login and get JWT token
------	-------------	-------------------------

■ *Budget APIs*

Method	Endpoint	Description
--------	----------	-------------

-----	-----	-----
-------	-------	-------

POST	/api/budget/set/	Set monthly budget
------	------------------	--------------------

GET	/api/budget/status/	Get real-time budget status
-----	---------------------	-----------------------------

■ *Orders APIs*

Method	Endpoint	Description
--------	----------	-------------

-----	-----	-----
-------	-------	-------

POST	/api/orders/create/	Create new order
------	---------------------	------------------

GET	/api/orders/list/	Get user orders
-----	-------------------	-----------------

■■■ Installation & Setup

1■■■ *Clone Repository*

```
git clone https://github.com/your-username/smart-budget-ecommerce.git
cd smart-budget-ecommerce
```

2■■■ *Run with Docker (Recommended)*

```
docker-compose up --build
```

Backend will run at:

```
http://localhost:8000
```

3■■■ *Run Manually (Without Docker)*

Backend

```
cd backend
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

Frontend

```
cd frontend
npm install
npm start
```

■ Budget Calculation Logic

```
If total_spent >= monthly_limit:  
    status = EXCEEDED  
Else if total_spent >= 80% of monthly_limit:  
    status = WARNING  
Else:  
    status = SAFE
```

■ Example Budget Response

```
{  
  "monthly_limit": 5000,  
  "total_spent": 4200,  
  "remaining": 800,  
  "status": "WARNING"  
}
```

■ Future Enhancements

- AI purchase prediction
- Category-wise budgeting
- Family shared budget wallet
- Graph-based analytics dashboard
- Payment gateway integration
- Cloud deployment (AWS / Render)
- CI/CD with GitHub Actions

■ Resume Worthy Description

Built a production-ready full-stack Smart Budget Monitoring System using Django, DRF, JWT authentication, PostgreSQL, and React with Dockerized deployment and modular service-layer architecture.

■■■ Author

Abhishek Sharma

B.Tech CSE

Full Stack Developer (Django + React)

■ License

This project is licensed under the MIT License.

■ Generate README PDF

Run the helper script to generate a PDF version of this README:

```
python scripts/generate_readme_pdf.py
```

The output file will be `README.pdf` in the project root.